Coding-Agent CLI Platform Research

Deep comparative report on Claude Code, OpenAI Codex, Gemini CLI, and Google Antigravity

Prepared for: designing and hardening a custom coding-agent CLI with skills, workflows, hooks, policies, and multi-agent execution

Scope and research note

This report prioritizes official documentation and official product sources first, then adds official repositories, GitHub issues, and community reports only where they reveal implementation friction or undocumented edge cases. Research state is current to 15 March 2026. Google Antigravity's direct docs pages were not fully machine-readable in the browsing environment at the time of research, so the official Google Codelab and official Google Developers Blog launch article were used as primary official proxies for deep feature extraction.

Research date: 15 March 2026

Deliverable: One consolidated PDF with comparative analysis, source register, and implementation recommendations for your own CLI

Method: Official docs tree review → official repos/blogs → issues/discussions for caveats

How this report is organized

The report is deliberately structured for implementation use, not just product description. The first pages give you the executive picture; the middle chapters dissect each platform; the final chapters turn those findings into a concrete design for your own CLI.

- **Executive summary.** The shortest version of the findings and the architectural conclusion.
- **Cross-platform matrix.** A fast scan of instruction files, skills, workflows, hooks, security, automation, and packaging.
- **Per-platform deep dives.** Detailed treatment of Claude Code, OpenAI Codex, Gemini CLI, and Google Antigravity.
- **Comparative analysis.** What converges, where the products differ, and which ideas are most portable.
- **Recommended architecture for your CLI.** A proposed file layout, precedence model, security model, hook/event surface, and rollout plan.
- **Source register and field notes.** Official pages reviewed plus secondary-source caveats.

> **Reading tip**
>
> If you are actively building a CLI right now, start with the executive summary, skim the cross-platform matrix, then jump straight to “Recommended architecture for your CLI.” Return to the platform chapters when you need exact product behavior or packaging inspiration.

Executive summary

All four products are solving the same core problem—how to let a coding agent act autonomously without becoming unpredictable—but they package the solution very differently. Claude Code is the most terminal-native and the most opinionated about hooks, plugins, and subagents. OpenAI Codex is the most complete end-to-end operating model, spanning CLI, app, IDE, SDK, app-server, worktrees, admin policy, and background automations. Gemini CLI is the strongest pure CLI control plane: it combines an open skill format, explicit extension packaging, a full hook surface, a policy engine, checkpointing, and serious headless/automation behavior. Google Antigravity is the most ambitious user-experience layer: it treats agent management as a first-class surface and centers trust through artifacts, review flows, browser verification, and asynchronous multi-agent orchestration.

The most important convergence is around instruction layering and progressive disclosure. Every platform separates always-on context from occasionally-needed expertise. Always-on guidance lives in files such as CLAUDE.md, AGENTS.md, GEMINI.md, or Antigravity rules. Specialized expertise lives in skill packages or saved workflows that are loaded only when relevant. For your own CLI, this separation is not optional: it is the most effective way to keep context windows small, reduce tool bloat, and preserve predictable behavior as the feature set grows.

The second major convergence is that the portable center of gravity is no longer “prompt templates” but package directories. Gemini CLI, OpenAI Codex, and Antigravity all orbit a SKILL.md-centered package model, typically with optional scripts, references, and assets. Claude Code uses a similar concept but extends it with richer frontmatter, direct slash invocation semantics, tool permissions, plugins, and subagent interactions. If you want your own CLI to interoperate with existing ecosystems, the highest-leverage move is to support an Agent-Skills-style directory layout out of the box and then add compatibility shims for vendor-specific metadata.

The third convergence is that serious agent products all separate at least three layers of autonomy control: interactive approvals, durable policy, and execution isolation. Gemini exposes this most cleanly at the CLI level through approval modes plus a TOML policy engine. Codex goes deeper with sandbox modes, rules, protected paths, admin requirements, and explicit security documentation around web search and prompt injection. Claude Code combines settings precedence, managed policy, hook controls, and tool permissions inside skills and subagents. Antigravity frames security more through terminal auto-execution modes, review policies, browser allowlists, and explicit user-review surfaces.

If your goal is a CLI that “works perfectly,” the winning architecture is not to imitate one platform wholesale. Instead, copy the best layer from each: adopt Codex’s progressive skill discovery and enterprise-grade config layering, Gemini’s hook and policy engine model, Claude’s plugin bundling and subagent ergonomics, and Antigravity’s artifact-first trust model for planning, code review, and verification.

Bottom-line recommendation

Design your CLI around four primitives: (1) layered instruction files, (2) SKILL.md packages with progressive disclosure, (3) a first-class hook and policy engine, and (4) isolated execution environments for unattended work. Then add subagents, background worktrees, and artifact generation on top of that base.

Key takeaways for a custom coding-agent CLI

- **Adopt a vendor-neutral skill package.** Use SKILL.md plus optional scripts/, references/, assets/, and an optional machine-readable metadata file for UI and dependencies.
- **Separate rules from workflows.** Rules are always-on guidance; workflows are on-demand saved prompts; skills are reusable capability packages. Do not collapse these into one feature.

- **Make invocation control explicit.** You need both implicit matching and explicit invocation, plus a per-skill way to disable one or both.
- **Treat hooks and policy as different layers.** Hooks observe or reshape behavior; policy decides whether tool calls are allowed.
- **Prefer worktree or sandbox isolation for unattended runs.** Background tasks, scheduled jobs, and CI sessions should not share the same mutable workspace as an interactive human session.
- **Generate artifacts, not just logs.** Plans, task lists, diff summaries, and walkthroughs build trust much faster than raw tool traces.

Cross-platform comparison matrix

This section compresses the four platforms into implementation-relevant dimensions. The goal is not product ranking. The goal is to identify which design ideas are portable, which are vendor-specific, and which combinations are strongest for a custom CLI.

Dimension	Claude Code	OpenAI Codex	Gemini CLI	Google Antigravity
Primary surface	Terminal-first coding agent with IDE / web / desktop touchpoints and deep CLI-native customization.	Full product stack: CLI, app, IDE, web, SDK, app-server, background automations, worktrees.	CLI-first platform with strong local config, extensions, policy, headless operation, and sandboxing.	Agent-first development platform with an Editor plus a Manager surface for asynchronous agent orchestration.
Always-on instructions	CLAUDE.md, memory, settings, output styles, and managed policy.	AGENTS.md / AGENTS.override.md hierarchy plus config and rules.	GEMINI.md hierarchy plus just-in-time context loading.	Global GEMINI.md rule plus workspace rule files.
On-demand prompts / commands	Skills and slash commands; legacy commands still supported but skills take precedence.	Slash commands and explicit skill mentions ($skill).	Custom commands in TOML plus explicit skill activation and slash-driven ops.	Workflows are saved prompts triggered with /.
Skill package model	Directory with SKILL.md plus optional supporting files; richer frontmatter and tool permissions.	Directory with SKILL.md plus scripts/references/assets and optional agents/openai.yaml.	Agent Skills standard with SKILL.md plus scripts/references/assets; workspace, user, and extension tiers.	SKILL.md-based directory packages in global or workspace scope; progressive disclosure emphasized.
Delegation model	Subagents plus experimental agent teams with mailbox, task list, and teammates.	Experimental multi-agent CLI plus app-side background execution and threads.	Experimental local and remote subagents; no full manager view.	Manager-first multi-agent orchestration across workspaces and tasks.
Hooks / lifecycle control	Very broad hook surface around session, prompt, tool, subagent, compacting, elicitation, and scheduling.	No comparable general hook bus in the docs; strongest controls live in rules, approvals, and app/server integrations.	Strong synchronous hook surface before/after model, tool selection, tools, compression, session, notifications.	Not documented as a generic hook framework; control is exposed mainly through workflows, artifacts, review, and settings.
Packaging / distribution	Plugins and plugin marketplaces can bundle skills, agents, hooks, MCP, and LSP metadata.	Skills can exist at repo/user/admin/system scopes; no equivalent marketplace-first packaging model in current docs.	Extensions bundle manifest, MCP, hooks, skills, commands, and subagents.	Rules, workflows, and skills are customizable, but preview-era public packaging/distribution is less developed.
MCP / external tools	First-class MCP support with	MCP servers configurable in	MCP servers in settings and	Documentation reviewed

Dimension	Claude Code	OpenAI Codex	Gemini CLI	Google Antigravity
	local/project/user scopes and managed controls.	config.toml; skills can declare dependencies for smoother setup.	extensions with auth flows and tool filtering.	emphasized editor/terminal/browser tools more than a broad MCP ecosystem.
Approvals / policy / sandbox	Settings precedence, tool permissions, managed restrictions, hook controls, and subagent permission modes.	Most explicit enterprise-grade model: sandbox modes, approval policies, rules, protected paths, admin requirements.	Approval modes, TOML policy engine, multiple sandbox backends, and trusted folders.	Terminal auto-execution modes, review policy, JavaScript policy, allow/deny lists, browser allowlist.
Automation / unattended use	Session-scoped scheduled tasks via /loop and cron tools; durable automation delegated elsewhere.	Strongest unattended model via app automations, worktrees, SDK, app-server, and non-interactive mode.	Strong CLI automation through headless mode, JSONL events, MCP, hooks, and checkpointing.	Async agent manager is the core product story, but public automation packaging is still preview-shaped.
Trust / verification UX	Human-readable CLI workflows, hooks, and explicit skills; less artifact-centric than Antigravity.	Reviewable diffs, worktrees, app controls, and strong operational visibility.	Good operator controls and logs; trust comes from policy, hooks, and checkpointing more than artifacts.	Best artifact-first trust model: task lists, implementation plans, walkthroughs, screenshots, browser recordings.
Best idea to borrow	Plugin bundling + rich hook system + subagent ergonomics.	Progressive disclosure + config layering + sandbox/rules/worktrees + SDK/app-server.	Hook bus + policy engine + extension system + headless JSONL.	Artifact-first asynchronous manager UX and explicit separation between rules and workflows.

Compatibility lens: what maps most directly into your own CLI

Your CLI primitive	Closest Claude concept	Closest Codex concept	Closest Gemini concept	Closest Antigravity concept
Global instructions	CLAUDE.md / memory	Global AGENTS.md	Global GEMINI.md	Global GEMINI.md rule
Project instructions	Project CLAUDE.md / settings	Nested AGENTS.md files	Workspace/ancestor GEMINI.md files	Workspace rules
Reusable skill package	Skill folder under .claude/skills	Skill folder under .agents/skills	Skill folder under .gemini/skills or .agents/skills	Skill folder under .agents/skills or ~/.gemini/antigravity/skills
On-demand saved prompt	Slash-invoked skill / custom	Slash commands or explicit $skill	Custom command TOML or explicit	Workflow invoked with /

Your CLI primitive	Closest Claude concept	Closest Codex concept	Closest Gemini concept	Closest Antigravity concept
	command		skill	
Delegated specialist	Subagent / teammate	Spawned agent	Local or remote subagent	Separate managed agent instance
Behavior interception	Hook	Rules / approvals / app integration	Hook	Review policy / artifacts / settings
External capability bundle	Plugin	MCP + skill + app/server integration	Extension	Not yet a mature public equivalent
Background job	Scheduled task (session only)	Automation / worktree / SDK job	Headless process / external scheduler	Manager-surface background agent

Cross-vendor architectural conclusion

The most reusable foundation is: Codex/Gemini-style SKILL.md packages + Gemini-style hooks/policy + Codex-style sandboxes/worktrees + Antigravity-style artifacts + Claude-style plugin bundling and subagent ergonomics.

Claude Code deep dive

Platform	One-line read	Strongest implementation ideas	Watch-outs	Research confidence
Claude Code	Terminal-native agent system with unusually rich hooks, plugins, subagents, and packaging for reusable workflows.	Broad hook surface, plugin bundling, skills integrated with commands, and strong delegation ergonomics via subagents and agent teams.	Skill format is less portable than the Agent Skills convention; some plugin/marketplace behaviors are still maturing in the field.	High

Claude Code is the platform in this comparison that feels most like a power-user terminal environment that grew a full extensibility system instead of a conventional IDE that added agent features later. The result is a product that is extremely interesting for CLI architects because it treats reusable behavior as a composition of skills, subagents, hooks, plugins, MCP servers, and settings scopes rather than as one monolithic “agent config” file.

1. Product model and mental model

Claude Code’s official documentation positions the product as a coding agent that can operate in the terminal and connect outward into IDEs, Claude surfaces, MCP servers, and third-party integrations. The feature set clusters around a few primitives: persistent context (CLAUDE.md and memory), reusable skills, deterministic hooks, custom subagents, and pluggable external capabilities.

The important architectural point is that Claude Code does not force you to choose between “prompt engineering” and “tooling.” It expects you to externalize durable behavior into files and packages. That makes it a strong reference design if you want your own CLI to feel scriptable and composable instead of fragile and chat-like.

2. Persistent context: CLAUDE.md, memory, and always-on guidance

Claude’s documentation recommends reserving always-relevant project guidance for CLAUDE.md and using skills for knowledge that is only occasionally relevant. This distinction matters. If a rule or convention is true for almost every task in a repo, put it in persistent context. If it is domain-specific, procedural, or expensive, package it as a skill instead.

The practical lesson for your CLI is to make “always-on” and “on-demand” context different storage classes. Users should be able to look at a file and know whether it will be loaded every turn or only on demand.

3. Skills and slash commands

Claude’s skills are stored as directories with a required SKILL.md file and optional supporting files. Discovery spans personal, project, and plugin scopes. The docs explicitly show personal skills in ~/.claude/skills/<name>/SKILL.md, project skills in .claude/skills/<name>/SKILL.md, and plugin-provided skills under the plugin root. Claude also discovers nested project skill directories, which is useful for monorepos.

What makes Claude’s skill model distinctive is that it fuses three concerns that other platforms often keep separate: reusable knowledge, direct slash invocation, and temporary permission elevation. A skill can be model-invoked implicitly, user-invoked explicitly, hidden from the user menu, restricted to

particular tools, run in a forked context, or tied to a specific agent. That is a very rich behavior layer for a simple Markdown package.

Claude skill frontmatter that matters most

Field	Purpose	Why it matters for a custom CLI
description	Tells Claude when the skill is relevant.	This is the minimum metadata for implicit invocation.
disable-model-invocation	Prevents automatic invocation by Claude.	You need this for side-effecting workflows such as deploy, release, or messaging actions.
user-invocable	Controls visibility in the slash menu.	This separates background knowledge from explicit user actions.
allowed-tools	Grants tool access without per-tool approval while the skill is active.	Excellent pattern for temporary capability envelopes tied to a skill.
context: fork	Runs the skill in a forked context.	Useful for isolation and for procedures that should not pollute the main thread.
agent / model / hooks	Lets a skill influence delegation, model choice, or hook behavior.	Shows how far a skill package can act as a mini capability profile rather than a text snippet.

Representative Claude skill pattern

```
---
name: deploy
description: Deploy the application to production
context: fork
disable-model-invocation: true
allowed-tools: Bash(git *), Bash(gh *), Read
---

Deploy $ARGUMENTS to production:
1. Run tests
2. Build artifacts
3. Execute deployment steps
4. Verify the release succeeded
```

Claude's invocation controls are especially instructive. The default state is that both the user and Claude can invoke a skill, and the description is always in context. Setting disable-model-invocation: true removes the skill from automatic model selection, making it suitable for command-like workflows. Setting user-invocable: false hides it from the user-facing slash menu while leaving it available to Claude as background knowledge. This is one of the clearest "capability gating" designs in the set.

Claude also supports arguments via placeholders such as $ARGUMENTS and positional forms. If your own CLI supports explicit workflows, argument placeholders are worth copying because they make skills command-like without inventing a totally separate command DSL.

4. Subagents and agent teams

Claude's subagents are file-based specialists defined in Markdown with YAML frontmatter. They can specify a prompt, allowed and disallowed tools, model, permission mode, MCP servers, hooks, max turns, startup skills, and memory. This is a very capable local specialist format: far richer than a single system-prompt file, but still human-editable.

The experimental agent teams feature takes this a step further. Claude can create a lead plus multiple teammates, maintain a shared task list, use mailboxes for inter-agent messaging, and coordinate work across sessions. The design is interesting because it treats multi-agent work as a local operating model: teammates are not just invisible internal tool calls; they are independent Claude sessions with explicit task states and collaboration mechanics.

Subagent capability	Claude behavior	Design lesson
Prompt + frontmatter	Markdown body becomes the specialist prompt; frontmatter controls tools/model/permissions.	Human-readable agent definitions are easier to review and version.
Permission modes	default, acceptEdits, dontAsk, bypassPermissions, and plan.	Permission behavior should be part of the agent definition, not just a global CLI flag.
Startup skills	Subagents can preload skills into context; they do not inherit parent skills automatically.	Make inheritance explicit. Hidden inherited context causes debugging pain.
Hooks	Subagents can carry hooks in frontmatter.	Hooks should compose at multiple levels: global, workflow, and specialist.
Teams	Experimental multi-session coordination with tasks and teammate communication.	If you build multi-agent support, give it explicit coordination structures rather than relying on prompt magic.

5. Hooks: arguably Claude Code's most differentiated feature

Claude's hook system is unusually broad. The docs cover events around session start and end, prompt submission, tool execution, permission requests, subagent lifecycle, instructions loading, compaction, worktree creation/removal, teammate state changes, and more. Hooks can be implemented as local commands, HTTP calls, prompt-style hooks, or agent-based hooks.

This breadth matters because it lets you solve both safety and workflow problems without changing the core agent loop. You can use hooks to annotate context, enforce quality gates, rewrite prompts, capture telemetry, block unsafe operations, or integrate with external control systems. For a custom CLI, this is one of the strongest patterns in the research set.

Event family	Examples	Best use in a custom CLI
Session lifecycle	SessionStart, SessionEnd, InstructionsLoaded, ConfigChange	Bootstrap context, compliance checks, audit setup.
Prompt lifecycle	UserPromptSubmit, Stop, Notification	Prompt rewriting, logging, routing, notification integration.
Tool lifecycle	PreToolUse, PostToolUse, PostToolUseFailure, PermissionRequest	Argument validation, policy bridging, telemetry, guardrails.
Agent lifecycle	SubagentStart, SubagentStop, TeammateIdle, TaskCompleted	Multi-agent coordination, quality gates, task orchestration.
Maintenance lifecycle	PreCompact, PostCompact, WorktreeCreate, WorktreeRemove, Elicitation*	Compaction hygiene, environment setup, human-input checkpoints.

What to copy

Claude's hook model is worth copying almost verbatim: broad event coverage, multiple hook backends, and the ability to restrict hooks through managed policy. Even if your first version only supports local command hooks, design the event taxonomy now so you do not paint yourself into a corner later.

6. Plugins, marketplaces, MCP, and packaging

Claude plugins are a packaging layer that can bundle commands, agents, skills, hooks, MCP configuration, and LSP metadata. The plugin.json manifest lives under .claude-plugin/plugin.json, while the actual content directories live at plugin root. Plugin skills are namespaced to avoid collisions. This is a clean distribution pattern because one package can ship a coherent capability set instead of forcing users to install five different files in five different locations.

The direct design lesson is that your CLI should distinguish between a reusable primitive and a distributable bundle. A skill is one primitive. A plugin or extension is the shippable unit that may include many primitives. Gemini makes the same move with extensions. Claude's docs are especially explicit about the directory layout and namespacing discipline.

Representative Claude plugin structure

```
my-plugin/
├── .claude-plugin/
│   └── plugin.json
├── skills/
│   └── code-review/
│       └── SKILL.md
├── agents/
├── hooks/
│   └── hooks.json
├── .mcp.json
└── .lsp.json
```

7. Settings precedence, permissions, and scheduled work

Claude's settings system is hierarchical and enterprise-aware. Managed settings sit above everything else and cannot be overridden. Below that are user and project scopes, with arrays often merging rather than replacing. This is the right pattern for organizations because it allows additive policy without forcing every team to duplicate a giant config file.

Scheduled tasks in Claude are intentionally lightweight and session-scoped. The /loop skill and underlying CronCreate / CronList / CronDelete tools are useful for polling and reminders inside an active session, but the docs are explicit that recurring tasks expire after three days, are subject to jitter, and disappear when the session exits. This is a strong example of good product boundaries: Claude does not pretend session cron is a production scheduler.

Area	Claude behavior	Implication for your CLI
Settings precedence	Managed settings override every other layer; arrays can merge.	Use explicit precedence and document merge vs replace rules per key.
Managed hook restrictions	Admins can allow only managed hooks and restrict allowed HTTP hook targets/env vars.	Security-critical extensibility must be admin-governable.
Skill permissions	allowed-tools grants scoped tool	Permission envelopes tied to

Area	Claude behavior	Implication for your CLI
	access while skill is active.	capability packages reduce constant approval friction.
Session scheduling	Up to 50 tasks, jittered timing, 3-day expiry, no persistence across restarts.	Keep lightweight session timers separate from durable automation systems.

8. What Claude Code does especially well

- **Best hook surface in the set.** It is broad enough to become the backbone of custom workflow automation and safety enforcement.
- **Excellent plugin bundling.** Skills, agents, hooks, MCP, and LSP can ship as one coherent package.
- **Strong operator ergonomics.** Subagents and agent teams give you a clear path from simple delegation to more structured multi-agent work.
- **Good capability gating.** disable-model-invocation, user-invocable, and allowed-tools let one skill behave like knowledge, a command, or a guarded workflow.

9. Main limitations and field notes

Claude's skill model is very powerful, but it is not the most portable. If your goal is cross-vendor ecosystem compatibility, the Gemini/Codex/Antigravity SKILL.md convention is closer to the center of gravity. Claude's additional frontmatter and command semantics are best treated as an adapter layer rather than the canonical neutral format.

Secondary sources also suggest that the plugin and marketplace story is still evolving in practice. GitHub issues reviewed during this research included reports about plugin hook behavior and marketplace skill visibility. These reports do not invalidate the official design, but they are a reminder to build strong debugging and introspection tools if you emulate this model.

10. Claude-inspired recommendations for your CLI

1. **Implement a first-class hook bus.** Do not treat hooks as an afterthought; they should be an architectural layer.
2. **Support a distributable bundle format.** Allow one package to ship skills, hooks, agents, and integration metadata.
3. **Let skills carry permission envelopes.** It is a clean way to reduce prompt friction without making the whole session unsafe.
4. **Make agent definitions file-based and reviewable.** Claude's Markdown + YAML subagent pattern is practical and transparent.
5. **If you add teams, give them real coordination objects.** Task lists, mailboxes, and explicit states beat hidden multi-agent magic.

OpenAI Codex deep dive

Platform	One-line read	Strongest implementation ideas	Watch-outs	Research confidence
OpenAI Codex	The most complete operating system for coding agents in this set: CLI, app, IDE, web, SDK, app-server, sandboxing, skills, rules, worktrees, and automations.	Progressive skill loading, AGENTS.md layering, sandbox/rules/admin controls, worktrees, and programmatic surfaces.	Some advanced features are still experimental or surface-specific; the breadth can create configuration complexity if copied wholesale.	High

Codex is the broadest platform in the comparison. It is not just a CLI and not just a coding assistant. The documentation describes a coherent stack that spans interactive terminal use, non-interactive execution, IDE integration, background automations, isolated worktrees, programmatic control, and rich client integration. If you are building a serious agent platform rather than just a terminal wrapper around an LLM, Codex is the strongest systems reference.

1. Product model: an agent platform, not just a terminal tool

The Codex docs are notable because they define multiple surfaces that all rest on a common execution model: a CLI, the Codex app, an IDE extension, web/cloud environments, an SDK, and an app-server. This means the platform has to solve not only prompting and tools, but also thread continuity, approvals, secure execution, background work, and embedded-client protocols.

For your own CLI, the lesson is strategic: decide early whether you are building only an interactive terminal experience or whether you are building a reusable agent runtime that can later power an IDE extension, background jobs, or internal product integrations. Codex clearly chose the second path.

2. Persistent instructions: AGENTS.md is a real system, not a slogan

Codex's AGENTS.md model is one of the clearest instruction-layering systems in any agent product. At global scope it reads AGENTS.override.md if present, otherwise AGENTS.md. At project scope it walks from repo root down to the current working directory, taking at most one instruction file per directory and concatenating them from broadest to most specific. It also supports fallback filenames and a configurable byte limit via project_doc_max_bytes.

That design solves three common problems cleanly: personal defaults, repository-wide guidance, and deeply local overrides. It also keeps instruction precedence easy to reason about. For your own CLI, this is probably the best reference model for hierarchical instruction discovery.

Representative Codex instruction layering

```
~/.codex/AGENTS.md
repo-root/AGENTS.md
repo-root/services/payments/AGENTS.override.md

# Discovery rule
# 1. Global AGENTS.override.md else global AGENTS.md
# 2. Walk from project root to current directory
# 3. In each directory: AGENTS.override.md, then AGENTS.md, then fallback names
# 4. Concatenate from root downward until byte cap is reached
```

Instruction mechanism	Codex behavior	Why it is useful

Global defaults	~/.codex/AGENTS.md or AGENTS.override.md	Gives a durable personal operating style.
Project overrides	Walks root → current directory and concatenates one file per directory.	Supports monorepos and team-local norms without giant files.
Fallback names	project_doc_fallback_filenames config option.	Lets teams reuse existing guide filenames.
Byte limit	project_doc_max_bytes, 32 KiB by default.	Prevents runaway prompt growth and forces deliberate structuring.
CODEX_HOME	Relocates the profile/home directory.	Very useful for automation users, CI, and profile separation.

3. Skills: progressive disclosure done right

Codex's skill design is one of the strongest pieces of architecture in the set. A skill is a directory with SKILL.md plus optional scripts, references, assets, and an optional agents/openai.yaml metadata file. At startup, Codex does not load every skill body into context. It starts with metadata—name, description, path, and optional UI metadata—and only loads the full SKILL.md when the skill is actually used. That is the cleanest documented explanation of progressive disclosure in the research set.

Discovery is also well-designed: Codex reads skills from repository, user, admin, and system locations. Repository skills live under .agents/skills and are discovered while walking upward from the working directory toward repo root. This gives you both local specialization and shared team capabilities without needing a central registry.

Representative Codex skill package

```
my-skill/
├── SKILL.md
├── scripts/
├── references/
├── assets/
└── agents/
    └── openai.yaml
```

Minimal Codex SKILL.md

```
---
name: security-review
description: Review diffs for security issues. Use for PR review or before deployment.
---

Follow this checklist:
1. Check auth / authz paths
2. Inspect secrets handling
3. Review unsafe deserialization and shell execution
```

Representative Codex agents/openai.yaml

```
interface:
  display_name: "Security Review"
  short_description: "Review changes for security issues"
  icon_small: "./assets/shield-small.svg"
  brand_color: "#3B82F6"
  default_prompt: "Run a security-focused review"

policy:
  allow_implicit_invocation: false
```

```
dependencies:
  tools:
    - type: "mcp"
      value: "github"
      description: "GitHub MCP server"
```

The optional openai.yaml file is also highly instructive. It separates user-facing metadata, invocation policy, and dependencies from the instructional body. That is a pattern worth stealing. Many CLIs overload SKILL.md with UI metadata, operational policy, and instructions all in one place. Codex shows a cleaner split.

A particularly good idea is allow_implicit_invocation. Some skills should only run when explicitly invoked, even if the description matches. Your own CLI should definitely support both implicit and explicit-only skills.

Skill capability	Codex behavior	Design lesson
Progressive loading	Metadata first; full SKILL.md only when needed.	This is the best documented token-discipline pattern in the set.
Multiple discovery tiers	Repo, user, admin, system.	Enterprise and team use both need first-class support.
Optional machine-readable metadata	agents/openai.yaml	Keep UI, policy, and dependencies separate from prose instructions.
Explicit + implicit invocation	/skills, $skill, or description-based matching.	Users need both discoverability and automation.
Built-in skill creation/install flows	$skill-creator and $skill-installer.	Your CLI should make packaging easy, not purely manual.

4. Rules, approvals, and sandboxing

Codex has the strongest formal security model in the comparison. The docs cover approval policies, sandbox modes, web search modes, protected paths, admin-enforced requirements, and executable command rules. This is not just “ask for approval sometimes.” It is a layered control system.

The rules language is especially interesting. Codex uses Starlark-based .rules files and provides codex execpolicy check to test how rules apply to a command. When multiple rules match, the most restrictive result wins. It can even split simple shell chains safely before evaluating them, then fall back to conservative whole-command evaluation for more complex shell constructs.

Control layer	Codex behavior	What to copy
Sandbox mode	read-only, workspace-write, danger-full-access, plus OS-specific implementations.	Make execution isolation a first-class mode, not a hidden flag.
Approval policy	on-request, untrusted, never, granular reject blocks.	Approval policy should be separate from sandbox mode.
Rules engine	Starlark .rules with prefix_rule, justification, and most-restrictive-wins semantics.	Use a deterministic policy language; do not bury safety logic in prompts.
Protected paths	.git, .agents, .codex remain read-only inside writable workspaces.	Protect your own control directories from accidental mutation.

Control layer	Codex behavior	What to copy
Web search modes	cached by default, live optional, disabled possible.	Treat browsing as a separate risk class from shell/network execution.

Codex's most important systems lesson

Do not merge sandboxing, approval prompts, and policy rules into one config switch. They solve different problems. Sandboxing limits what commands can do, approvals mediate autonomy, and rules encode durable organizational policy.

5. Multi-agents and role-based delegation

Codex's multi-agent support is documented as experimental, but the architecture is already quite revealing. Enabling features.multi_agent exposes collaboration tools such as spawn_agent, send_input, resume_agent, wait, close_agent, and spawn_agents_on_csv. The docs also describe a monitor role for long-running commands or polling workflows.

This is a powerful pattern because it treats multi-agent work as workflow orchestration rather than merely "let the model think harder." It also implies your own CLI should separate the control plane for agent spawning and coordination from the high-level prompt layer.

Multi-agent design choice	Codex approach	Takeaway
Feature gating	Experimental and explicitly enabled.	Advanced autonomy should be opt-in while it matures.
Tool surface	Dedicated collaboration tools.	Multi-agent systems need explicit coordination primitives.
Role diversity	Different models/instructions per agent; monitor role for waiting/polling.	Roles should be first-class, not hidden prompt text.
Surface limitations	CLI-visible today; app/IDE visibility still evolving.	Document feature parity by surface to avoid operator confusion.

6. Automations, worktrees, local environments, SDK, and app-server

Codex is the only product in this research set with a genuinely comprehensive unattended-work model in the official docs. The app can schedule automations that run in the background, add findings to an inbox, archive no-op runs, and execute either in the local project or in a dedicated worktree. Those automations can explicitly invoke skills.

Worktrees are a major design advantage. Codex stores them under CODEX_HOME/worktrees, can move threads between local checkout and worktree, and can restore snapshots when needed. This is exactly the kind of operational hygiene that a mature coding-agent platform needs once background tasks become normal.

The SDK and app-server then expose the runtime programmatically. The TypeScript SDK is for controlling Codex in CI or inside your own applications. The app-server is for deep client integrations with approvals, history, and streamed agent events via JSON-RPC-like protocols over stdio or WebSocket. Together, these surfaces demonstrate how an agent runtime can be reused without making the CLI itself the only integration point.

Capability	Codex support	Design value
Background automations	Recurring app-side tasks with inbox/triage behavior.	Shows how to make unattended work operationally visible.
Worktrees	Dedicated isolated Git workspaces under CODEX_HOME.	Strong default for any background code-modifying task.
Local environments	Setup scripts and common actions per project.	Codify environment preparation instead of relying on prompts.
SDK	@openai/codex-sdk for programmatic control.	Lets your agent runtime become a platform, not just a terminal UI.
App-server	JSON-RPC-like protocol with approvals and streamed events.	Ideal reference if you later build IDE or internal tool integrations.

7. What Codex does especially well

- **Best overall systems architecture.** It combines context layering, skills, security, worktrees, automations, and programmatic surfaces into one coherent model.
- **Best progressive disclosure story.** Skills are clearly token-efficient and the docs explain why.
- **Best enterprise posture.** Sandboxing, rules, protected paths, admin requirements, and security docs are unusually mature.
- **Strongest "runtime, not just UI" design.** The SDK and app-server make the agent engine reusable beyond the terminal.

8. Main limitations and field notes

The main downside of Codex's breadth is complexity. It has many knobs, many surfaces, and some features are still marked experimental or not yet fully surfaced across CLI, app, and IDE. If you copy the Codex model wholesale without ruthless simplification, your own CLI could become difficult to understand.

Secondary sources reviewed during research also point to some edge-case friction around approvals, sandbox behavior, and extension parity. That does not undermine the architecture; it reinforces the need for good introspection commands, clear default modes, and strong surface-parity documentation.

9. Codex-inspired recommendations for your CLI

6. **Use AGENTS.md-style hierarchical instructions.** It is the clearest model for global + project + local guidance.
7. **Adopt progressive skill loading by default.** Do not inject every skill body into every prompt.
8. **Split prose instructions from machine metadata.** Use SKILL.md for behavior and a structured metadata file for UI/policy/dependencies.
9. **Protect your own control directories.** Your .cli, .agents, and repo metadata should not be casually writable even in editable sandboxes.
10. **Build worktree isolation before shipping unattended automation.** Otherwise background agents will eventually corrupt an active local checkout.
11. **If you plan future IDE integration, design an event protocol now.** Codex app-server is a strong model for streamed events and approvals.

Gemini CLI deep dive

Platform	One-line read	Strongest implementation ideas	Watch-outs	Research confidence
Gemini CLI	The strongest pure CLI control plane in the set: open skills, hooks, policy engine, extensions, checkpointing, and headless automation.	A very complete local customization model with skills, custom commands, hooks, extensions, MCP, policy tiers, and headless JSONL.	Some agent features remain experimental, and the system can feel sprawling if you enable every customization path at once.	High

Gemini CLI is the product I would study first if I were optimizing specifically for a powerful terminal-native agent that remains transparent, extensible, and automation-friendly. It offers fewer polished managerial surfaces than Antigravity and fewer cross-surface runtime layers than Codex, but as a command-line operating environment it is exceptionally rich.

1. Product model: a serious local CLI with first-class customization

Gemini CLI's documentation tree is unusually explicit about local customization. Skills, context files, custom commands, hooks, extensions, policy rules, MCP, subagents, sandboxing, and headless execution are all documented as first-class features rather than obscure extras. This tells you that Google expects users to shape the CLI heavily.

For your own CLI, that is a powerful signal: if you want serious users, treat customization as product surface, not as hidden config. Gemini's docs read like a local operator's manual, and that is exactly what many advanced coding-agent users want.

2. Context and memory: GEMINI.md is the cleanest hierarchical model after Codex

Gemini uses GEMINI.md as its main instruction file and loads it hierarchically. It reads a global file from ~/.gemini/GEMINI.md, then workspace and ancestor GEMINI.md files, and then just-in-time context files discovered when tools access a file or directory. That just-in-time layer is especially notable: it means highly local instructions do not need to be loaded until work actually touches that part of the tree.

This is a very smart design for large monorepos. It gives you broad project guidance without forcing every tiny package's instructions into the model on every turn. If your CLI will operate in large repos, copying this JIT context behavior is worth serious consideration.

Context layer	Gemini behavior	Why it matters
Global	~/.gemini/GEMINI.md	Reusable personal defaults across all work.
Workspace / ancestor	Searches workspace directories and parent directories for GEMINI.md	Lets instruction scope follow repo structure.
Just-in-time	Loads GEMINI.md files near files/directories actually being accessed	Prevents unnecessary prompt growth in large trees.
Manual controls	/memory show, reload, add	Makes context inspection and repair much easier for operators.

3. Agent Skills: open and highly portable

Gemini's Agent Skills implementation is one of the clearest references for a portable skill ecosystem. Skills live in workspace, user, or extension scopes, use a SKILL.md file with required name and description, and can include scripts, references, and assets. The docs explicitly call out discovery precedence—Workspace over User over Extension—and even note that .agents/skills is preferred over .gemini/skills within a tier to stay compatible with other agent tools.

That last point is extremely important for your own CLI. Gemini is effectively endorsing a tool-agnostic skills path convention. If you support .agents/skills and standard SKILL.md packages, you will be naturally compatible with more ecosystems.

Representative Gemini skill package

```
my-skill/
├── SKILL.md
├── scripts/
├── references/
└── assets/
```

Representative Gemini SKILL.md

```
---
name: code-review
description: Reviews code changes for bugs, style issues, and best practices.
---

# Code Review Skill

When reviewing code, follow this checklist:
1. Correctness
2. Edge cases
3. Style
4. Performance
```

Gemini documents the activation flow clearly: it scans discovery tiers, injects names and descriptions of enabled skills into the system prompt, and when a skill matches it uses the activate_skill tool and presents a consent prompt showing the skill's name, purpose, and directory path. This is a beautifully transparent design: the model gets discoverability, the user gets visibility, and the system avoids loading every skill body up front.

As a design takeaway, your own CLI should expose skill discovery and activation as explicit runtime objects. Do not make users wonder why the model suddenly knows something. A visible activation step dramatically improves trust.

Skill design point	Gemini approach	Design lesson
Discovery precedence	Workspace > User > Extension; .agents/skills preferred alias.	Build compatibility paths deliberately.
Activation tool	activate_skill with explicit consent prompt.	Make skill activation visible to the operator.
Open package shape	SKILL.md + scripts/references/assets.	Simple directory packages scale well.
Management commands	/skills list/link/disable/enable/reload plus gemini skills ...	Operators need inspection and lifecycle commands.

4. Custom commands: a separate primitive from skills

Gemini's custom commands are an especially good reminder that not every reusable action should be a skill. Commands live in .gemini/commands or ~/.gemini/commands, are stored as TOML, and focus on prompt templates plus dynamic injections such as {{args}}, shell command blocks !{...}, and file/directory content injections @{...}.

This is a very useful separation of concerns. A command is a user-facing shortcut. A skill is reusable model-invoked expertise. They are related, but they are not the same thing. Many CLIs blur them together; Gemini keeps them distinct.

Representative Gemini custom command TOML

```
description = "Generates a fix for a given issue."
prompt = """
Please provide a code fix for the issue described here: {{args}}.

Search results:
!{grep -R {{args}} .}
"""
```

5. Hooks: Gemini's control plane is unusually strong

Gemini's hooks run synchronously inside the agent loop and can intervene before and after model calls, before tool selection, before tool execution, after tool execution, around session boundaries, and around context compression. The hooks take structured JSON input and produce structured results, with clear exit-code semantics for success, warning, and system block behavior.

This is one of the most reusable pieces of design in the whole study. It is clean, deterministic, and clearly separated from higher-level policies. Hooks can add context, filter tools, validate arguments, redact content, log activity, or block actions, while the policy engine handles durable allow/deny decisions.

Hook timing	Typical use	Why it is powerful
BeforeModel / AfterModel	Modify prompts, swap models, redact outputs, log interactions.	Lets you control reasoning context and response handling.
BeforeToolSelection	Filter or bias available tools.	Excellent place to reduce bad tool choices without editing core logic.
BeforeTool / AfterTool	Validate arguments, block dangerous commands, log results.	Most practical place for workflow and safety enforcement.
PreCompress	Manage what survives context compression.	Critical once sessions get long.
SessionStart / SessionEnd / Notification	Bootstrap and integration behavior.	Turns the CLI into an evented system.

What to copy

Gemini's hook/event model is probably the best direct reference if you want a clean CLI-native interception system. Pair it with a documented stdin/stdout contract and explicit exit semantics. That makes hooks scriptable, testable, and enterprise-friendly.

6. Policy engine, approval modes, and sandboxing

Gemini's policy engine is another standout feature. Rules are expressed in TOML, evaluate tool calls based on conditions such as tool name or command prefix, and return allow, deny, or ask_user decisions. Higher-priority rules win, with tiered precedence across default, extension, workspace, user, and admin policy sources.

This is a superior design to a single global allow/deny list because it scales. You can scope policy to approval mode, MCP tools, command patterns, or workspace needs. The docs also explicitly recommend using deny rules instead of legacy tool exclusion settings. That kind of deprecation guidance is a sign of a maturing control plane.

On approvals, Gemini exposes default, auto_edit, and plan modes in settings, while YOLO is restricted to CLI flags. That is another smart safety boundary. Sandboxing is flexible too: Docker/Podman containers, gVisor/runsc, and LXC/LXD are all part of the documented story, which makes Gemini particularly interesting for Linux-heavy or container-first teams.

Control area	Gemini behavior	Design lesson
Approval modes	default, auto_edit, plan; YOLO only via CLI flag.	Keep the most dangerous autonomy modes harder to enable.
Policy engine	allow / deny / ask_user with priority tiers and mode scoping.	A CLI should have a real policy DSL, not just ad hoc prompts.
Wildcard tool matching	Supports built-in and MCP wildcard patterns.	Policies must understand external tool namespaces too.
Sandbox backends	Containers, runsc, LXC/LXD, and related config.	Execution isolation should be pluggable by environment.
Trusted folders	Separate trust concept for workspace behavior.	Trust state should not be an invisible assumption.

7. Extensions: packaging, config, and ecosystem shape

Gemini's extension system is excellent from a packaging perspective. Extensions live in ~/.gemini/extensions and are defined by gemini-extension.json. An extension can bundle MCP servers, context file naming, hooks, skills, custom commands, subagents, themes, and configurable settings. That is a broad but coherent bundle format.

Compared with Claude, the main difference is naming and manifest structure, not fundamental concept. Both products distinguish a reusable primitive (skill, hook, command) from a distributable unit (plugin, extension). If you build your own CLI, follow that pattern. Otherwise distribution and team sharing will eventually get messy.

Representative Gemini extension manifest

```
{
 "name": "my-extension",
 "version": "1.0.0",
 "description": "My awesome extension",
 "mcpServers": {
  "my-server": {
   "command": "node",
   "args": ["${extensionPath}/my-server.js"],
   "cwd": "${extensionPath}"
  }
 }
```

```
}
```

8. Subagents, remote agents, checkpointing, and headless mode

Gemini's subagents are explicitly experimental, but the design is promising. Local subagents are Markdown files with YAML frontmatter including name, description, kind, tools, model, temperature, and max_turns. Remote subagents use the Agent-to-Agent protocol and support authentication blocks aligned with the A2A security specification.

Checkpointing is another strong operational feature. Gemini keeps a shadow Git repository in its history area, storing snapshots, conversation history, and tool calls. This is a practical way to make long sessions recoverable and auditable.

Headless mode is also well thought through. It supports prompt input, output formats, streaming JSONL event types, output schemas, and clear exit codes. For a CLI that may later be wired into CI or shell pipelines, this is the right standard.

Capability	Gemini support	Why it matters
Local subagents	Markdown + YAML specialists.	A transparent way to delegate focused work.
Remote subagents	A2A-compatible remote agents with auth blocks.	Extends the CLI into a networked agent fabric.
Checkpointing	Shadow Git repo storing snapshots and session data.	Operational resilience and auditability.
Headless mode	Structured output, JSONL streaming, schema support, exit codes.	Exactly what automation and CI users need.
Todos / plan artifacts	Native support around plan mode and task steering.	Improves long-task operator control.

9. What Gemini CLI does especially well

- **Best pure CLI customization stack.** Skills, commands, hooks, policies, extensions, and headless mode all feel like first-class citizens.
- **Very portable skill story.** The .agents/skills alias and standard SKILL.md package shape are strategically important.
- **Excellent control plane.** Hooks and policy together provide a strong local governance model.
- **Strong automation posture.** Headless JSONL, checkpointing, and sandboxing are highly practical.

10. Main limitations and field notes

Gemini's biggest challenge is sprawl. There are many ways to customize behavior, and not every user will know when to reach for skills, commands, hooks, policies, or extensions. If you borrow heavily from Gemini, invest in documentation and operator introspection commands.

Subagents and remote agents are still experimental, and community discussion reviewed during the research suggests some rough edges when subagents and skills interact in complex flows. That is not unusual for early multi-agent features, but it is a reminder to keep delegation features observable.

11. Gemini-inspired recommendations for your CLI

12. **Support a neutral .agents/skills path.** This is the highest-leverage interoperability choice you can make.

13. **Separate skills from user shortcuts.** Keep skill packages distinct from command templates or workflows.
14. **Implement a real hook contract.** Structured stdin/stdout plus clear exit semantics will pay dividends quickly.
15. **Add a policy engine early.** Even a small TOML/YAML policy layer is better than hard-coded approval logic.
16. **Design headless mode as a product, not an afterthought.** Streaming JSONL and stable exit codes turn a CLI into infrastructure.
17. **Use JIT context loading for large repos.** It keeps prompts lean without losing locality.

Google Antigravity deep dive

Platform	One-line read	Strongest implementation ideas	Watch-outs	Research confidence
Google Antigravity	The most manager-centric and artifact-driven product in the set: an editor plus a mission-control surface for asynchronous agents.	Artifact-first trust model, browser-integrated validation, and explicit separation between rules, workflows, and skills.	Still preview-era; direct docs accessibility was weaker during research, so some conclusions rely on Google's official codelab and official launch article rather than the docs site alone.	Medium-high

Antigravity is the outlier in the set because it is not primarily a terminal product. It is an agent-first development platform that elevates management of asynchronous agents to the top-level user experience. That makes it especially valuable if your long-term ambition is bigger than a CLI—if you eventually want an orchestrator surface, background agents, verification artifacts, and browser-driven validation loops.

1. Product model: Editor + Agent Manager

Google's official codelab describes Antigravity as an agentic development platform that forks the VS Code foundation but reorganizes the experience around two windows: the Editor and the Agent Manager. The Agent Manager acts as mission control, where users can spawn, monitor, and interact with multiple agents operating asynchronously across different workspaces or tasks.

This is the most explicit "manager surface" in the research set. Codex app automations and Claude agent teams both move in this direction, but Antigravity makes it the headline concept. For your own CLI, the lesson is not that you need a GUI today. The lesson is that asynchronous agent work eventually needs a control plane separate from the conversational surface.

Surface	What it does	Why it matters architecturally
Agent Manager	Launches tasks, tracks parallel agents, shows artifacts and approval requests.	A control plane for asynchronous work.
Editor	Traditional code editing plus inline AI commands and agent side panel.	Human-in-the-loop surface for direct intervention.
Browser integration	Lets agents verify behavior in a controlled browser through a specialized subagent.	Turns validation into a first-class workflow rather than an external manual step.

2. Artifacts: Antigravity's signature design choice

Antigravity's most differentiated idea is that it uses artifacts as the main trust surface. The official sources describe task lists, implementation plans, walkthroughs, code diffs, screenshots, and browser recordings as first-class outputs. Users can comment on them Google-Docs-style, and the agent incorporates the feedback.

This is an important conceptual shift. Many coding agents make the operator verify raw tool logs or diffs alone. Antigravity turns progress and verification into explicit deliverables. That is a major usability insight for any future agent platform, including a CLI if you are willing to generate plan files, verification notes, and review artifacts inside the repo.

Artifact type	Purpose	Portable lesson for a CLI
Task list	Shows the work decomposition before coding.	Persist planned steps in a reviewable format.
Implementation plan	Documents intended architecture and change strategy.	Generate a human-readable plan before deep edits.
Code diffs	Standard reviewable code change surface.	Keep diffs central but not the only trust surface.
Walkthrough	Explains how the result was verified.	Produce a test/verification narrative after major changes.
Screenshots / browser recordings	Demonstrate UI or workflow behavior.	For web/UI agents, proof of execution is as important as code.

What to copy

If your CLI will ever touch product code, tests, deployment flows, or UI behavior, generate artifacts by default for complex tasks: plan.md, checklist.md, diff-summary.md, and verify.md. Antigravity shows that this dramatically improves trust without requiring users to read every raw trace.

3. Rules, workflows, and skills: three separate primitives

The official codelab is especially useful here because it clearly differentiates three concepts. Rules are always-on guidance. Workflows are saved prompts triggered on demand with /. Skills are specialized packages of knowledge that stay dormant until a request matches their description. This separation is excellent product design.

The storage layout is also practical. The codelab documents a global rule in ~/.gemini/GEMINI.md, global workflows in ~/.gemini/antigravity/global_workflows/, workspace rules in <workspace>/.agents/rules/, workspace workflows in <workspace>/.agents/workflows/, global skills in ~/.gemini/antigravity/skills/, and workspace skills in <workspace>/.agents/skills/.

Representative Antigravity customization layout

```
# Global and workspace customization paths

~/.gemini/GEMINI.md
~/.gemini/antigravity/global_workflows/<workflow>.md
~/.gemini/antigravity/skills/<skill>/SKILL.md

<workspace>/.agents/rules/
<workspace>/.agents/workflows/
<workspace>/.agents/skills/<skill>/SKILL.md
```

Representative Antigravity skill pattern (from official codelab)

```
---
name: code-review
description: Reviews code changes for bugs, style issues, and best practices.
---

# Code Review Skill
```

```
1. Check correctness
2. Check edge cases
3. Check style
4. Suggest improvements
```

Antigravity's skill format looks closest to the same SKILL.md package family used by Gemini and Codex. The official codelab explicitly calls out progressive disclosure: the skill is only loaded into context when the request matches the skill's description. Optional scripts, references, and assets are part of the package model.

For your CLI, that means Antigravity reinforces the case for a neutral SKILL.md-centered ecosystem. By 2026, that shape is clearly not a one-off. It is becoming a cross-product pattern.

Primitive	Antigravity meaning	Design guidance
Rule	Always-on behavioral guidance.	Equivalent to system instructions or persistent project conventions.
Workflow	Saved prompt invoked with / on demand.	Good for user-triggered routines that should not always be in context.
Skill	Dormant specialist package loaded only when needed.	Best for reusable, targeted expertise with optional resources.

4. Browser-integrated verification and planning modes

The official sources emphasize Antigravity's close browser integration. When the agent needs to interact with the browser, it invokes a browser subagent running a model specialized for open pages. The browser subagent can click, scroll, type, read console logs, inspect the DOM, take screenshots, parse pages as markdown, and record video.

Antigravity also distinguishes Planning and Fast modes. Planning produces richer artifacts and more deliberate task grouping; Fast is for quick localized tasks. This is not just a UX toggle. It reflects a broader principle that the autonomy and verbosity profile of an agent should be adjustable per task.

Capability	Official Antigravity description	What to steal
Planning mode	Agent plans before executing; better for deep research, complex tasks, collaborative work.	Add a deliberate high-reasoning mode that produces artifacts.
Fast mode	Direct execution for simple localized tasks.	Offer a low-friction path for obvious tasks.
Browser subagent	Specialized model and tools for browser operation.	Treat browser control as a specialist, not just another generic tool.
Feedback on artifacts	Comments on plans, diffs, screenshots, recordings.	Let review happen on outputs, not only in chat.

5. Security and autonomy controls

The codelab gives a reasonably concrete picture of Antigravity's safety controls despite the preview nature of the product. Setup flows expose terminal execution policy, review policy, and JavaScript execution policy. Later sections describe terminal command auto-execution modes Off, Auto, and Turbo, as well as allow lists, deny lists, and a browser URL allowlist file under HOME/.gemini/antigravity/browserAllowlist.txt.

This is a simpler model than Codex's sandbox-and-rules architecture, but it still encodes an important idea: shell autonomy, artifact review, and browser script execution are different trust dimensions and should be configurable separately.

Control	Antigravity model	Takeaway for your CLI
Terminal auto execution	Off / Auto / Turbo.	Users need a high-level autonomy dial for shell actions.
Allow / deny lists	Command-based safelists and blocklists.	Useful supplement even when you also have richer policies.
Review policy	Always proceed / agent decides / request review.	Review gating should be distinct from shell execution gating.
Browser allowlist	Explicit trusted-domain file for browser access.	Web browsing deserves its own risk boundary.
JavaScript execution policy	Separate control for browser JS execution.	Browser interaction should not be one undifferentiated permission.

6. Packaging and ecosystem maturity

Antigravity is still a preview product, and the public packaging story is not as mature or as explicitly documented as Claude's plugins or Gemini's extensions. The reviewed official sources focus much more on workflows, skills, browser use, artifact review, and manager/editor UX than on a broad public extension ecosystem.

That is not necessarily a weakness for the product, but it does mean you should avoid assuming that Antigravity is already the best reference for plugin distribution. Use it instead as the strongest reference for asynchronous orchestration UX and artifact-centered trust.

7. What Antigravity does especially well

- **Best manager-surface vision.** It treats asynchronous multi-agent work as a primary workflow, not a hidden extra.
- **Best artifact-centered trust model.** Plans, walkthroughs, screenshots, and recordings give users proof instead of just promises.
- **Excellent conceptual separation.** Rules, workflows, and skills are distinct, which keeps the product model legible.
- **Strong browser-validation story.** Browser work is not bolted on; it is integrated into the verification loop.

8. Main limitations and confidence note

Antigravity's main limitation in this report is documentation accessibility and product maturity. The direct docs site was not fully readable in the browsing environment during research, so this chapter leans on Google's official codelab and official blog post as primary official sources. That means the conceptual model is clear, but there is less line-by-line low-level config detail than in the Claude, Codex, and Gemini sections.

Because the product is preview-stage, you should also expect some behavior, packaging conventions, or surface details to evolve quickly. Use Antigravity as a source of product architecture and UX ideas more than as a locked-down compatibility target.

9. Antigravity-inspired recommendations for your CLI

18. **Generate artifacts by default for complex tasks.** This is Antigravity's biggest product insight.
19. **Separate manager-style orchestration from interactive editing.** Even in a CLI, background jobs and active chat should not feel identical.
20. **Keep rules, workflows, and skills distinct.** Conflating them makes a product hard to understand and govern.
21. **Treat browser validation as a specialized delegated workflow.** A browser agent deserves different controls and evidence than shell commands.
22. **Use planning modes intentionally.** Complex tasks should be able to produce more structure, not just more tokens.

Comparative analysis: what actually converges across the platforms

The most important conclusion from the research is that the platforms are not converging on one identical product surface, but they are converging on a shared architecture. That architecture has a small number of recurring layers: a persistent instruction stack, a reusable skill package system, explicit autonomy controls, specialist delegation, and some form of packaged distribution.

Once you look at the products through that lens, the noise falls away. The file names differ, the UI differs, and the brand language differs, but the underlying shapes are increasingly stable. That is good news for you, because it means you can build a custom CLI that is both principled and interoperable rather than merely copying one vendor's quirks.

1. The instruction stack is now a core product primitive

All four platforms distinguish broad, durable guidance from narrow, on-demand expertise. Claude uses CLAUDE.md and memory. Codex uses AGENTS.md and overrides. Gemini uses GEMINI.md with just-in-time locality. Antigravity separates global/workspace rules from workflows and skills.

The design implication is simple: your CLI needs an instruction stack, not a single global prompt file. At minimum, support user-global, project-root, nested local, and optionally per-task overlays. Make merge order explicit. Also add a command that tells the user exactly which instruction files are active.

2. SKILL.md packages are becoming the portable center of gravity

Gemini, Codex, and Antigravity all present a very similar skill package idea: a directory containing SKILL.md plus optional resources. Codex and Antigravity explicitly highlight progressive disclosure. Gemini explicitly points toward cross-tool compatibility through the .agents/skills alias. Claude's skill system is richer and slightly more vendor-specific, but it still validates the same basic idea of package directories over inline prompt snippets.

This means your safest compatibility bet is to make SKILL.md the canonical package format in your CLI. You can then add vendor-specific adapters for Claude-style fields or optional machine metadata files. If you instead invent a completely different format, you will be working against the direction of the ecosystem.

3. Rules, workflows, commands, and skills should stay separate

One of the easiest mistakes when designing an agent CLI is to create a single "macro" or "prompt pack" feature that tries to cover every use case. The platforms show that this is a bad idea. Rules are always-on. Workflows and custom commands are user-triggered routines. Skills are reusable capabilities that the model can often invoke automatically. Subagents are delegable specialists. Hooks and policies sit under those layers as governance and interception surfaces.

The products that keep these concepts distinct—especially Gemini and Antigravity—have a much cleaner mental model than systems that over-collapse them.

Primitive	Purpose	Recommended storage	Invocation pattern
Instruction file	Always-on durable guidance.	~/.yourcli/AGENTS.md and nested repo files.	Loaded automatically by precedence.
Rule / policy	Deterministic allow/deny/ask behavior.	~/.yourcli/policies/*.toml or *.star	Evaluated automatically on tool calls.
Workflow / command	User-triggered saved	.yourcli/workflows/*.md	Explicit /workflow or

Primitive	Purpose	Recommended storage	Invocation pattern
	prompt or routine.	and commands/*.toml	/command.
Skill	Reusable specialist package with progressive disclosure.	.agents/skills/<name>/SKILL.md	Implicit by description and/or explicit by name.
Subagent	Delegable specialist actor.	.yourcli/agents/*.md	Spawned by the main agent or explicitly selected.
Hook	Event-based interception and integration.	.yourcli/hooks/hooks.json	Runs on lifecycle events.

4. Sandboxing, approvals, and policy are different control layers

Codex and Gemini are strongest here, with Claude also reinforcing the separation. Sandboxing defines what commands can technically do. Approvals define when the human must confirm. Policy defines which operations are categorically allowed, denied, or gated under certain conditions. Hooks can participate but should not replace policy.

If your CLI only has one big flag such as --yolo or --safe, it will not scale. Users need to choose, for example, read-only + no prompts for CI, workspace-write + on-request for interactive coding, or workspace-write + policy-gated background runs for scheduled automation.

5. Background work eventually demands isolation

Claude's session-scoped scheduler is intentionally limited. Gemini's headless mode assumes you will often use the CLI under an external scheduler. Codex formalizes worktree isolation and app automations. Antigravity formalizes asynchronous agent work at the UX level. The common message is clear: once work continues after the user stops actively watching, it should not be running inside the same mutable context as the operator's live editing session.

For your CLI, that means isolated worktrees, container sandboxes, or branch-based clones should be part of the roadmap before you market unattended automation heavily.

6. Trust is moving from logs to artifacts

Antigravity pushes this furthest, but all platforms are moving toward more structured trust surfaces: reviewable diffs, explicit plans, checkpointing, or worktree separation. The broader lesson is that raw tool logs do not scale as the only verification mechanism. People need summarized, inspectable, commentable outputs.

Even in a text-only CLI, you can implement this today. Write plan.md, tasks.md, diff-summary.md, and verify.md into a known artifacts directory for long tasks. That gives your users something better than "the agent says it finished."

Synthesis in one sentence

The ecosystem is standardizing on layered instructions + progressive skill packages + explicit autonomy controls + specialist delegation + reviewable artifacts.

Recommended architecture for your own coding-agent CLI

This section turns the research into a concrete proposal. It is intentionally opinionated. The goal is not to imitate a single vendor. The goal is to give you a design that is internally coherent, interoperable with the direction of the ecosystem, and practical for a CLI that you want to work reliably in real projects.

1. Core design principles

23. **Use a layered instruction system.** Global defaults, repo-wide guidance, nested local overrides, and per-run overlays should be separate and inspectable.
24. **Make SKILL.md the canonical reusable package.** This is the best cross-vendor compatibility choice.
25. **Keep reusable primitives separate.** Instruction files, skills, workflows/commands, policies, hooks, and agents should not collapse into one format.
26. **Make autonomy controls composable.** Sandbox mode, approval policy, and tool policy must be independent controls.
27. **Treat background work as a different execution class.** Use worktrees or isolated sandboxes for unattended tasks.
28. **Generate artifacts for complex tasks.** Plans, task lists, diff summaries, and verification notes should be first-class outputs.
29. **Build introspection in from day one.** Users need commands that answer: what instructions are loaded, what skills are available, why a skill activated, why a tool was blocked, and which policies won.

2. Proposed filesystem layout

Recommended storage layout

```
# user-global
~/.yourcli/
├── config.toml
├── AGENTS.md
├── skills/
│   └── <skill-name>/
│       ├── SKILL.md
│       ├── scripts/
│       ├── references/
│       └── assets/
├── workflows/
│   └── <workflow>.md
├── commands/
│   └── <command>.toml
├── agents/
│   └── <agent>.md
├── hooks/
│   └── hooks.json
├── policies/
│   └── default.toml
└── plugins/
    └── <plugin>/

# project-local
<repo>/
├── AGENTS.md
```

```
├── .yourcli/
│   ├── config.toml
│   ├── hooks/hooks.json
│   ├── policies/project.toml
│   ├── workflows/
│   ├── commands/
│   ├── agents/
│   └── plugins/
└── .agents/
    └── skills/
        └── <skill-name>/
            └── SKILL.md
```

The key compatibility choice here is deliberate: skills live under .agents/skills in repositories, because that is the most ecosystem-aligned path across Codex, Gemini, and Antigravity. Global skills can live under ~/.yourcli/skills, but I recommend also supporting ~/.agents/skills as an alias if you want frictionless import/export from other tools.

I also recommend using AGENTS.md as your neutral always-on instruction filename, while optionally importing or aliasing CLAUDE.md and GEMINI.md. AGENTS.md is the most semantically neutral name in the current ecosystem and works well for a CLI that may later extend beyond coding into broader agent workflows.

3. Instruction discovery and precedence

Precedence layer (highest first)	What it contains	Merge behavior
Run-time overrides	CLI flags, profile selection, explicit per-run instruction overlays.	Replace scalars; append certain arrays.
Managed/admin policy	Enterprise-enforced config, allowed integrations, forbidden autonomy modes.	Hard override; cannot be relaxed locally.
Project-local config	<repo>/.yourcli/config.toml and nested AGENTS.md files.	Project-specific defaults and instructions.
Workspace-local policies/hooks	<repo>/.yourcli/policies and hooks.	Mergeable where safe; explicit priority for rules.
User config	~/.yourcli/config.toml, ~/.yourcli/AGENTS.md, global hooks and policies.	Baseline defaults.
Bundled defaults	Built-in skills, templates, and default policies.	Lowest priority.

Copy Codex's instruction discovery pattern almost directly: walk from project root to current directory, taking one instruction file per directory, and concatenate from broadest to narrowest. Add a configurable byte cap, and expose the active chain with a command such as /status or yourcli instructions show.

For configuration keys, explicitly document whether a key merges or replaces. Arrays such as allowed paths or enabled MCP servers often need additive merging. Scalar values such as model or approval mode should usually be last-writer-wins unless they are admin-enforced.

4. Recommended skill spec

Use a neutral SKILL.md format for the instructional body, plus an optional metadata sidecar when you need UI/policy/dependency data. This follows the ecosystem trend while staying friendly to direct editing.

Recommended neutral SKILL.md

```
---
name: security-review
description: Review diffs or code changes for security issues. Use for PR review, pre-release checks, or incident analysis.
version: 1
tags: [security, review]
explicit_only: false
user_visible: true
context_mode: on-demand
allowed_tools: [read_file, grep_search]
---

# Security Review

## Goal
Review the relevant changes for security issues.

## Inputs
- Diff or changed files
- Relevant config and secret-handling code

## Procedure
1. Check auth / authz changes
2. Check secrets and credentials handling
3. Check shell execution, deserialization, and network calls
4. Summarize findings by severity

## Output
- Findings grouped by severity
- Recommended fixes
- Residual risk notes
```

Optional machine-readable skill metadata

```
interface:
  display_name: "Security Review"
  short_description: "Review changes for security issues"
  icon: "./assets/shield.svg"

policy:
  allow_implicit_invocation: true
  approval_envelope: "read_only"

dependencies:
  mcp:
    - github
  scripts:
    - "./scripts/review.sh"

compat:
  import_aliases:
    - "code-review"
    - "security-audit"
```

Recommended field	Why include it	Borrowed from
name / description	Minimum for discovery and invocation.	All skill ecosystems.
explicit_only or allow_implicit_invocation	Critical for side-effecting or delicate workflows.	Codex + Claude.
user_visible	Lets a skill exist as background knowledge without appearing as a command.	Claude.
allowed_tools / approval_envelope	Temporary capability control tied to the skill.	Claude + Codex principles.
version / tags	Makes long-term maintenance and filtering easier.	General packaging best practice.
metadata sidecar	Separates UI/dependencies from prose.	Codex openai.yaml and Gemini/extension manifest mindset.

Do not put too much machine logic into the Markdown frontmatter. Keep the body human-readable and instructional. If you need dependency declarations, UI metadata, or admin flags, move those into a sidecar YAML/TOML/JSON file. That keeps the neutral skill package portable and easier to review.

Also support skill import adapters. For example, when importing a Claude skill, map disable-model-invocation to explicit_only, user-invocable to user_visible, and allowed-tools to allowed_tools. When importing a Gemini or Codex skill, you will usually only need to honor name, description, and the directory layout.

5. Workflows and commands

Workflows should be saved prompts triggered explicitly, closer to Antigravity workflows or Gemini custom commands than to model-invoked skills. They are for routines the user consciously chooses to run: release prep, write changelog, review PR, summarize branch, or prepare an incident note.

Commands are a lighter-weight user shortcut layer. Keep them user-facing. Let them support argument substitution, safe shell injection with confirmation, and file-content injection. This is the part of Gemini worth copying almost exactly.

Recommended workflow markdown

```
# workflows/release-check.md
name: release-check
description: Run the project's release-readiness process.
mode: planning

1. Build a release checklist
2. Run the configured validation skill set
3. Produce release-check.md and diff-summary.md
```

Recommended command TOML

```
# commands/grep-code.toml
description = "Summarize matches for a search pattern"
prompt = """
Summarize the findings for {{args}}.

Search results:
!{rg -n {{args}} .}
"""
```

6. Agents and delegation

Subagents should be separate from skills. A skill is reusable expertise. An agent is an actor with its own tool budget, prompt, and permission profile. Borrow Claude's and Gemini's Markdown+frontmatter approach here. It is simple, versionable, and easy to inspect.

You do not need full "agent teams" in version 1. But you should design for them. At minimum, give your runtime internal concepts for spawned agent thread, role, instructions, tool scope, max turns, and result handoff. That way you can add teams later without rewriting everything.

Recommended agent definition

```
---
name: security-auditor
description: Investigates security-sensitive changes and risky code paths.
tools: [read_file, grep_search, diff, run_tests]
model: reasoning-high
approval_mode: plan
max_turns: 8
startup_skills: [security-review]
---

You are a focused security auditor.
Stay narrow.
Return findings with severity, evidence, and recommended fix.
```

Agent field	Purpose	Recommended default
name / description	Discovery and routing.	Required.
tools	Per-agent capability scope.	Narrow allowlist.
model	Reasoning / cost profile.	Inherit from session unless specified.
approval_mode	Permission behavior for the agent.	Plan or on-request for most specialists.
max_turns	Prevents runaway loops.	Small positive integer.
startup_skills	Loads relevant expertise immediately.	Optional and explicit.

7. Hooks and the event bus

Borrow the event taxonomy from Claude and Gemini. You want coverage at session boundaries, prompt submission, model request/response, tool selection, pre-tool, post-tool, post-tool-failure, permission request, compaction, agent start/stop, and notification. If you later add background jobs or teams, extend the same bus rather than inventing a second one.

Hook execution should be deterministic and structured. I recommend local-command hooks first, with JSON payloads over stdin and JSON responses over stdout. Then add optional HTTP hooks and internal scripted hooks later. This gives you a stable, testable contract.

Recommended hooks manifest

```
{
  "version": 1,
  "hooks": [
    {
      "event": "BeforeTool",
      "matcher": "run_shell_command",
      "command": "~/.yourcli/hooks/validate_shell.py"
```

```
    },
    {
      "event": "AfterTool",
      "matcher": "*",
      "command": "~/.yourcli/hooks/log_tool_result.py"
    }
  ]
}
```

Event	What it should expose	Primary use
SessionStart / SessionEnd	Workspace, profile, instruction chain, trust state.	Bootstrap and audit.
BeforeModel / AfterModel	Prompt metadata, model, tokens, response summary.	Prompt shaping, redaction, logging.
BeforeToolSelection	Tool candidates and context.	Filter or rank tools.
BeforeTool	Tool name, args, actor, skill/agent source.	Validation and safety checks.
AfterTool / AfterToolFailure	Tool result or error summary.	Telemetry, retries, artifacts.
PermissionRequest	Requested escalation and reason.	Custom approval flows.
AgentStart / AgentStop	Role, parent thread, assigned skills.	Multi-agent coordination.
PreCompact / PostCompact	Context pruning summary.	Long-session hygiene.

8. Policy, approvals, and sandboxing

This is where you should be most deliberate. Use three separate mechanisms: sandbox mode, approval policy, and tool policy. Sandbox mode controls technical capability. Approval policy controls whether the user must confirm. Tool policy controls durable allow/deny/ask decisions by tool, command prefix, argument pattern, or external integration namespace.

A TOML policy engine is likely the fastest path to something understandable and powerful. Gemini's model is a good reference. If you need stronger enterprise administration later, move toward a Starlark-style rules layer like Codex for more expressive command policy. But do not start by hiding everything in prompts.

Recommended mode	Behavior	Use case
read-only	Read files only; no edits, no shell mutation, no network without explicit allow.	Codebase exploration, CI summarization.
workspace-write	Edit inside workspace; restrict external paths and network by policy.	Default interactive coding mode.
workspace-write + on-request	Editable but asks for risky actions or external access.	Default human-in-loop mode.
workspace-write + never (background)	No prompts; still governed by policy and sandbox.	Background automations in isolated worktrees.
danger-full-access	No sandbox and minimal gating.	Only for explicit expert opt-in with loud warnings.

Recommended policy TOML

```
[[rule]]
toolName = "run_shell_command"
commandPrefix = "git push"
decision = "ask_user"
priority = 100

[[rule]]
toolName = "run_shell_command"
commandPrefix = "rm "
decision = "deny"
priority = 200
deny_message = "Deletion commands require manual execution"

[[rule]]
toolName = "mcp_github_*"
decision = "allow"
priority = 50
modes = ["workspace_write"]
```

9. Background jobs, worktrees, and artifacts

Use Codex as the main reference here. Interactive sessions can run in the main checkout. Background jobs should default to a detached worktree or equivalent isolated workspace. The job should produce artifacts into a known directory and, if it modifies code, summarize the diff before asking the human to review or merge.

You can still support session-scoped reminders like Claude's /loop, but keep them clearly separate from durable automations. The product boundary should be visible: session jobs are lightweight convenience features; unattended jobs are isolated, reviewable, and ideally resumable.

Recommended artifact output directory

```
artifacts/
├── task-list.md
├── implementation-plan.md
├── diff-summary.md
├── verify.md
├── screenshots/
└── logs/session.jsonl
```

10. Compatibility strategy

External format	What to support natively	What to translate
Codex skill	SKILL.md package shape, .agents/skills path, optional sidecar metadata import.	UI metadata and dependency declarations into your sidecar format.
Gemini skill	SKILL.md package shape and .agents/skills / .gemini/skills discovery.	Extension-provided packaging into your plugin model.
Claude skill	SKILL.md body and basic metadata import.	disable-model-invocation, user-invocable, allowed-tools, context/agent fields into your neutral equivalents.
Antigravity skill	SKILL.md package shape and workspace/global scope mapping.	Workflow/rule storage into your workflow and instruction systems.

External format	What to support natively	What to translate
Instruction files	AGENTS.md plus optional import of CLAUDE.md and GEMINI.md.	Resolve precedence and duplicate guidance carefully.

Interoperability is a force multiplier. Supporting import/export for existing skill directories means your users can bring working expertise with them instead of rewriting everything. That dramatically lowers adoption friction.

Do not aim for perfect semantic parity across vendors in version 1. Aim for a clear compatibility contract: what is imported losslessly, what is adapted, and what requires manual review.

11. Phased rollout plan

Phase	Ship in this phase	Why it belongs here
Phase 1: solid core	AGENTS.md layering, SKILL.md packages, explicit workflows/commands, basic approvals, read-only/workspace-write sandboxes, /status introspection.	This gets you a trustworthy daily-use CLI fast.
Phase 2: power-user control	Hooks, policy engine, plugin bundles, agent definitions, headless JSONL, artifact generation.	This turns the CLI into an extensible platform.
Phase 3: unattended automation	Worktrees, background jobs, setup scripts, resumable runs, CI/SDK integration.	Only add this once isolation and introspection are strong.
Phase 4: orchestrated agents	Multi-agent roles, task graph, mailbox/task states, manager surface if desired.	Teams and orchestration are valuable, but expensive to get right.

If you only copy four things

Copy Codex's progressive skill loading, Gemini's hooks and policy engine, Claude's plugin/subagent model, and Antigravity's artifact-first review loop.

Implementation checklist and anti-patterns

This section is meant to be actionable while you are building. It distills the research into concrete acceptance criteria and things to avoid.

1. Minimum viable agent CLI that is still trustworthy

- **Instruction discovery.** User-global AGENTS.md plus repo-root and nested AGENTS.md support.
- **Skill system.** SKILL.md packages under .agents/skills with explicit listing and activation visibility.
- **Workflow/command layer.** At least one explicit saved-prompt mechanism separate from skills.
- **Permission model.** read-only and workspace-write modes, plus on-request vs never approvals.
- **Status/introspection.** A command that shows active instructions, active profile, available skills, and sandbox/approval state.
- **Artifact output.** For complex tasks, emit a plan and a diff summary into a predictable location.

2. Hardening checklist for a serious v1

- **Policy engine.** Allow/deny/ask rules by tool, command prefix, argument pattern, and mode.
- **Hooks.** Structured before/after tool and model hooks plus session lifecycle hooks.
- **Protected paths.** Make .yourcli, .agents, and repo metadata read-only inside editable sandboxes.
- **Worktree isolation.** Background edits should not default to the live checkout.
- **Skill provenance.** Show users where a skill came from and why it activated.
- **Admin controls.** Support managed restrictions for the most dangerous capabilities, especially hooks and external tool integrations.

3. Anti-patterns to avoid

Anti-pattern	Why it is dangerous	Better pattern
One giant prompt file	Becomes unreadable, hard to scope, and too expensive to inject every turn.	Use layered instruction files plus on-demand skills.
Using skills as commands, rules, and policies all at once	Creates a confusing mental model and poor governance.	Keep skills, workflows, hooks, and policies separate.
Global YOLO as a normal default	Unsafe and hard to reason about in team environments.	Make high-autonomy modes explicit, loud, and opt-in.
No explanation of why a tool was blocked	Users will mistrust the product and disable safeguards.	Return policy justifications and status explanations.
Background jobs in the live workspace	Eventually corrupts unfinished work or surprises users.	Use worktrees or isolated sandboxes.
Invisible activation of huge context packs	Causes token blowups and unpredictable behavior.	Use progressive disclosure and visible activation events.
Vendor-locked custom package format	Makes ecosystem reuse difficult and raises migration cost.	Center your ecosystem on SKILL.md-compatible packages.

4. My recommended order of implementation effort

30. **Instruction layering and /status inspection first.** Users must understand what is loaded.
31. **Neutral SKILL.md ecosystem second.** This unlocks reusable workflows and future interoperability.

32. **Policy and sandbox separation third.** Without this, automation and team use will stay fragile.
33. **Hooks fourth.** Once the core is stable, hooks let you extend without refactoring the runtime.
34. **Artifacts fifth.** They improve trust quickly for complex tasks and multi-step work.
35. **Worktrees and background jobs sixth.** Add unattended execution only after strong controls exist.
36. **Multi-agent orchestration last.** It is valuable, but easier to build once every lower layer is already explicit.

Appendix A. Official source register

This appendix lists the main official pages and official companion sources reviewed while building the report. It is not a raw browser log; it is an implementation-oriented register of the pages that informed the analysis.

Claude Code official sources

Overview: https://code.claude.com/docs/en/overview

Features overview: https://code.claude.com/docs/en/features-overview

How Claude Code works: https://code.claude.com/docs/en/how-claude-code-works

Memory: https://code.claude.com/docs/en/memory

Best practices: https://code.claude.com/docs/en/best-practices

Settings: https://code.claude.com/docs/en/settings

CLI reference: https://code.claude.com/docs/en/cli-reference

Sub-agents: https://code.claude.com/docs/en/sub-agents

Agent teams: https://code.claude.com/docs/en/agent-teams

Hooks: https://code.claude.com/docs/en/hooks

Skills / slash commands: https://code.claude.com/docs/en/slash-commands

Output styles: https://code.claude.com/docs/en/output-styles

Plugins: https://code.claude.com/docs/en/plugins

Plugin marketplaces: https://code.claude.com/docs/en/plugin-marketplaces

Scheduled tasks: https://code.claude.com/docs/en/scheduled-tasks

MCP: https://code.claude.com/docs/en/mcp

Third-party integrations: https://code.claude.com/docs/en/third-party-integrations

Status line: https://code.claude.com/docs/en/statusline

OpenAI Codex official sources

Codex overview: https://developers.openai.com/codex/

Prompting: https://developers.openai.com/codex/prompting

Customization: https://developers.openai.com/codex/customization

Sandboxing: https://developers.openai.com/codex/sandboxing

Workflows: https://developers.openai.com/codex/workflows

CLI overview: https://developers.openai.com/codex/cli

CLI features: https://developers.openai.com/codex/cli/features

CLI reference: https://developers.openai.com/codex/cli/reference

Slash commands: https://developers.openai.com/codex/cli/slash-commands

Config basics: https://developers.openai.com/codex/config

Advanced config: https://developers.openai.com/codex/advanced-config

Config reference: https://developers.openai.com/codex/config-reference

Rules: https://developers.openai.com/codex/rules

AGENTS.md: https://developers.openai.com/codex/guides/agents-md

MCP: https://developers.openai.com/codex/mcp

Skills: https://developers.openai.com/codex/skills

Multi-agents: https://developers.openai.com/codex/multi-agent

Agent approvals & security: https://developers.openai.com/codex/agent-approvals-security

Non-interactive mode: https://developers.openai.com/codex/non-interactive

Codex SDK: https://developers.openai.com/codex/sdk

Codex app-server: https://developers.openai.com/codex/app-server

Codex app overview: https://developers.openai.com/codex/app

Automations: https://developers.openai.com/codex/app/automations

Worktrees: https://developers.openai.com/codex/app/worktrees

Local environments: https://developers.openai.com/codex/app/local-environments

IDE extension: https://developers.openai.com/codex/ide

Web overview: https://developers.openai.com/codex/web

Security overview: https://developers.openai.com/codex/security

Gemini CLI official sources

Docs home: https://geminicli.com/docs/

Creating Agent Skills: https://geminicli.com/docs/cli/creating-skills/

Agent Skills: https://geminicli.com/docs/cli/skills/

Manage context and memory: https://geminicli.com/docs/cli/context-and-memory/

GEMINI.md: https://geminicli.com/docs/cli/gemini-md/

Settings: https://geminicli.com/docs/cli/settings/

Custom commands: https://geminicli.com/docs/cli/custom-commands/

Hooks: https://geminicli.com/docs/hooks/

Hooks reference: https://geminicli.com/docs/extensions/hooks-reference/

MCP servers: https://geminicli.com/docs/core/mcp/

Plan Mode: https://geminicli.com/docs/core/plan-mode/

Subagents: https://geminicli.com/docs/core/subagents/

Remote Subagents: https://geminicli.com/docs/core/remote-agents/

Sandboxing: https://geminicli.com/docs/core/sandboxing/

Checkpointing: https://geminicli.com/docs/core/checkpointing/

Headless mode: https://geminicli.com/docs/core/headless/

Policy engine: https://geminicli.com/docs/reference/policy-engine/

Configuration reference: https://geminicli.com/docs/reference/configuration/

Extensions overview: https://geminicli.com/docs/extensions/

Build extensions: https://geminicli.com/docs/extensions/build/

Extension best practices: https://geminicli.com/docs/extensions/best-practices/

Extension reference: https://geminicli.com/docs/extensions/reference/

Google Antigravity official sources

Docs entrypoint: https://antigravity.google/docs/get-started

Official site: https://antigravity.google/

Download page: https://antigravity.google/download

Official codelab: Getting Started with Google Antigravity: https://codelabs.developers.google.com/getting-started-google-antigravity

Official blog: Build with Google Antigravity: https://developers.googleblog.com/en/build-with-google-antigravity-our-new-agentic-development-platform/

Antigravity documentation note

The direct Antigravity docs entrypoint was part of the official source set but was not fully parseable in the browsing environment during research. The official Google Codelab and official Google Developers Blog article were therefore used as the primary official feature references for detailed analysis.

Appendix B. Secondary-source field notes

Secondary sources were used only after the official documentation review. They were treated as evidence for field friction, ecosystem direction, or undocumented edge cases—not as replacements for official product behavior. Treat these notes as operational signals rather than canonical specs.

Claude Code

Official repo: https://github.com/anthropics/claude-code

Official skills repo: https://github.com/anthropics/skills

GitHub issue: plugin UserPromptSubmit hooks never execute: https://github.com/anthropics/claude-code/issues/10225

GitHub issue: marketplace skills not exposed through skill tool: https://github.com/anthropics/claude-code/issues/10568

Community discussion: hooks usage and workflow patterns: https://www.reddit.com/

- **Observed pattern.** Community and issue reports suggest that plugin hooks and marketplace-based skill exposure can still produce debugging friction in some scenarios.
- **How to use this signal.** If you copy Claude's plugin model, build excellent diagnostics for hook loading, skill discovery, and namespacing.

OpenAI Codex

Official repo: https://github.com/openai/codex

Official skills repo: https://github.com/openai/skills

Community issue: approval policy inconsistency in VS Code extension: https://github.com/openai/codex/issues

Community issue: sandbox/out-of-workspace edge case: https://github.com/openai/codex/issues

Community discussions on approvals/network/sandbox defaults: https://www.reddit.com/

- **Observed pattern.** Reports clustered around approval/sandbox edge cases and surface differences rather than around the core architecture.
- **How to use this signal.** Make surface parity and policy introspection explicit in your own CLI.

Gemini CLI

Official repo: https://github.com/google-gemini/gemini-cli

Historical issue: comprehensive hook system request / evolution: https://github.com/google-gemini/gemini-cli/issues/9070

Community discussion: subagents + skills rough edges: https://www.reddit.com/

- **Observed pattern.** Most rough edges discussed publicly center on newer multi-agent / subagent flows rather than the core hooks/skills/policy model.
- **How to use this signal.** Keep advanced delegation features clearly marked and observable while they mature.

Antigravity

Official blog coverage: https://developers.googleblog.com/en/build-with-google-antigravity-our-new-agentic-development-platform/

Community discussion on artifacts and workflow: https://www.reddit.com/

Secondary product coverage for launch framing: https://www.theverge.com/

- **Observed pattern.** Discussion focused more on artifact experience and early-preview workflow impressions than on a broad third-party extension ecosystem.
- **How to use this signal.** Treat Antigravity chiefly as UX inspiration for orchestration and verification.

Final conclusion

If I were building your CLI from scratch today, I would standardize on AGENTS.md-style layered instructions, a SKILL.md-compatible package ecosystem, Gemini-style hooks and policies, Codex-style worktree isolation and programmatic surfaces, Claude-style plugin/subagent composition, and Antigravity-style artifact generation for trust. That combination gives you the best balance of ecosystem compatibility, local operator control, safety, and future product headroom.

The most important decision is to resist building a single all-purpose customization mechanism. Keep instructions, skills, commands, hooks, agents, and policies distinct. Once those layers are clean, the product becomes much easier to reason about, document, test, and extend.

That is the consistent lesson across the four platforms: the best coding-agent systems are not just better prompts. They are better operating models.

> **Practical starting point**
>
> If you want one concrete next step, implement these three files first in your own CLI: AGENTS.md, .agents/skills/<name>/SKILL.md, and .yourcli/policies/default.toml. Those three primitives will force the right architecture much earlier than building more commands or adding more models.